

Machine Learning

Lesson 3

Linear Regression · Regularization · Loss Functions · Feature Selection

Part 1 Linear Regression · **Part 2** Regularization & Loss Functions · **Part 3** Feature Selection

Today's Agenda

Part 1

Linear Regression

Simple & Multiple LR · Assumptions · Categorical Variables · Dummy Variable Trap · Multicollinearity · PPS

Part 2

Regularization & Loss

LASSO · Ridge · Elastic Net · Robust Regression · MSE · RMSE · MAE · R^2 · Adjusted R^2

Part 3

Feature Selection

Occam's Razor · Filtering Methods · Wrapper Methods · Backward/Forward/Bidirectional · Advanced Strategies

Part 1

Linear Regression

Simple Linear Regression

The Model

$$\hat{y} = \beta_0 + \beta_1 \cdot x$$

$\hat{y}$ Predicted (dependent) variable

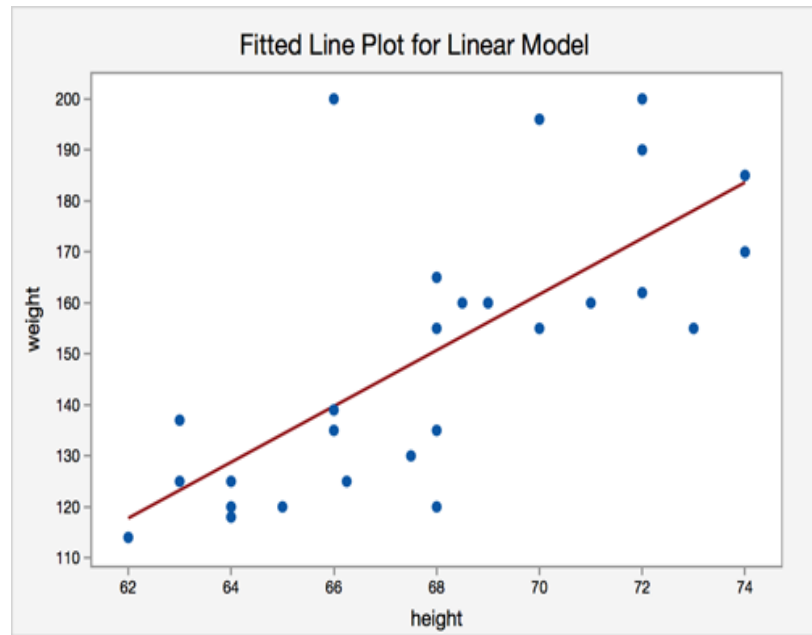
x Input (independent) variable

β_0 Intercept (value when $x = 0$)

β_1 Slope / regression coefficient

Goal: fit the line that minimises the **sum of squared residuals** (OLS).

Fitted Line — Height vs. Weight



Multiple Linear Regression

$$\hat{y} = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_n x_n$$

Venture Capital Profit Prediction Example

R&D Spend

Research & development investment

Administration

Overhead & admin costs

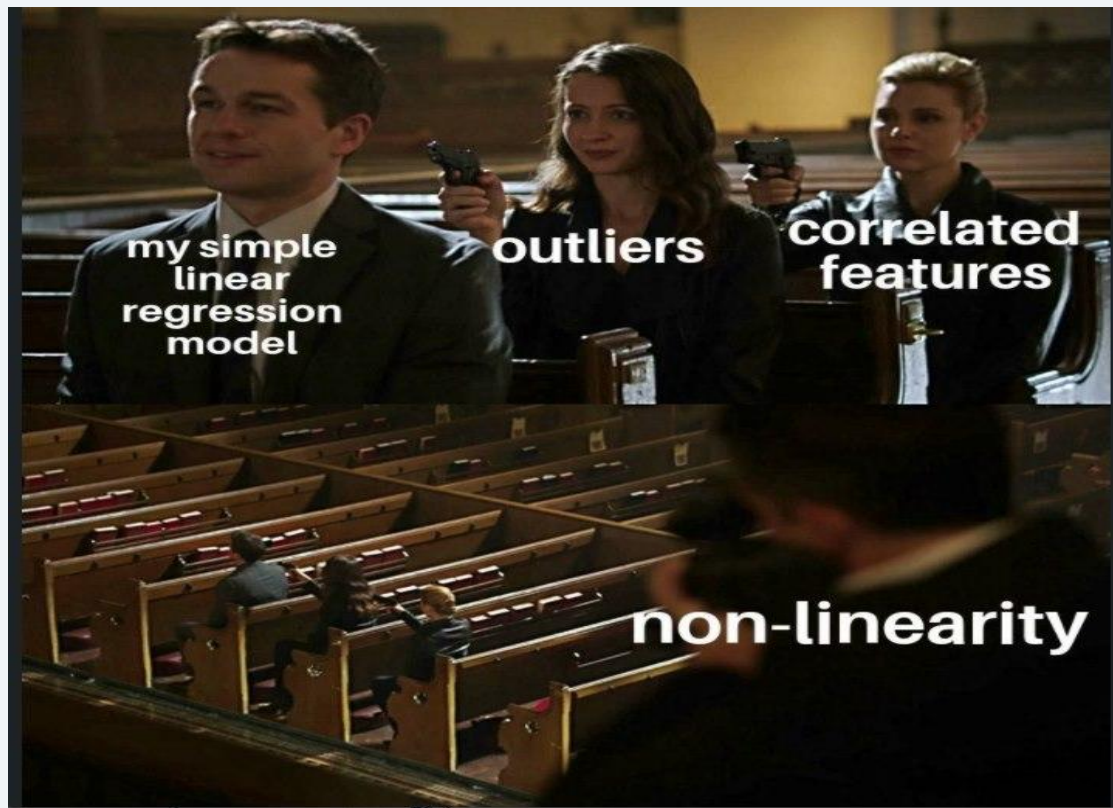
Marketing

Marketing budget

State (NY/CA/FL)

Categorical location

Threats to Simple Linear Regression



Outliers

Single extreme values can drag the regression line significantly off course.

Correlated Features

Multicollinearity inflates coefficient variance and makes estimates unreliable.

Non-Linearity

Forcing a line through curved data systematically biases predictions.

The 5 Assumptions of Linear Regression

1 Linearity

Relationship between X and Y must be linear

2 Homoscedasticity

Error variance is constant across all values of X

3 Independence of Errors

Residuals are uncorrelated with each other

4 Normality of Errors

Residuals must be approximately normally distributed

5 No Multicollinearity

Predictors must not be linearly dependent on each other

Reference: [towardsdatascience.com](https://towardsdatascience.com/the-pragmatist's-guide-to-assumptions-in-linear-regression/) — The Pragmatist's Guide to Assumptions in Linear Regression

Assumption 1 – Linearity

What it means

The expected value of Y must be a **linear function** of each predictor, holding others fixed.

How to test:

Scatter plot of each X vs. Y — look for a straight-line trend.

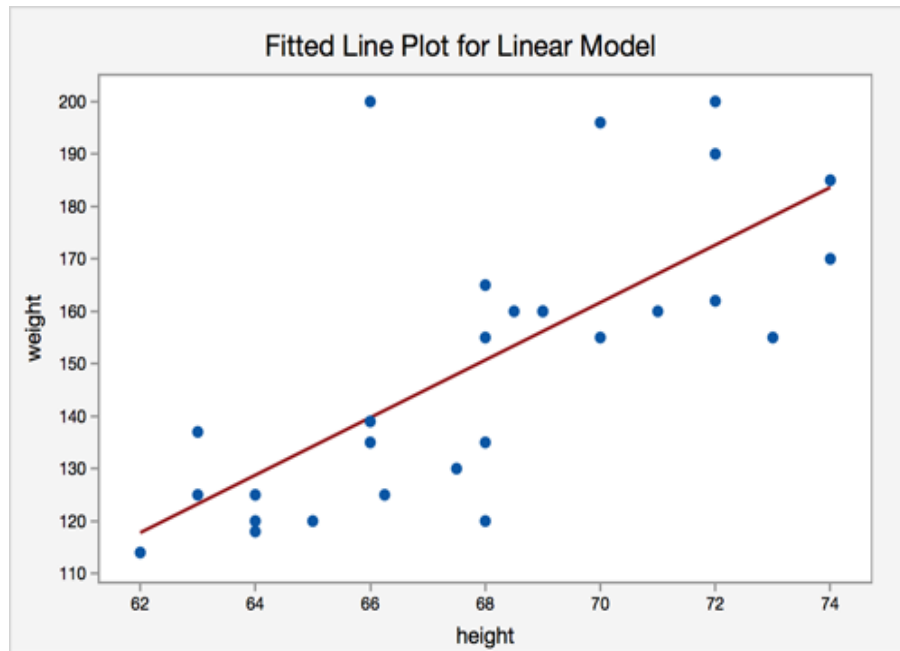
Height → Weight example:

The relationship is approximately linear — OLS is reasonable here.

⚠ Violation?

Consider polynomial features or tree-based models.

Fitted Line Plot (Height vs. Weight)



Assumption 2 – Homoscedasticity

Definition

Homoscedasticity means the error term has the **same variance** across all values of X.

Heteroscedasticity (the violation) occurs when error size differs across X values.

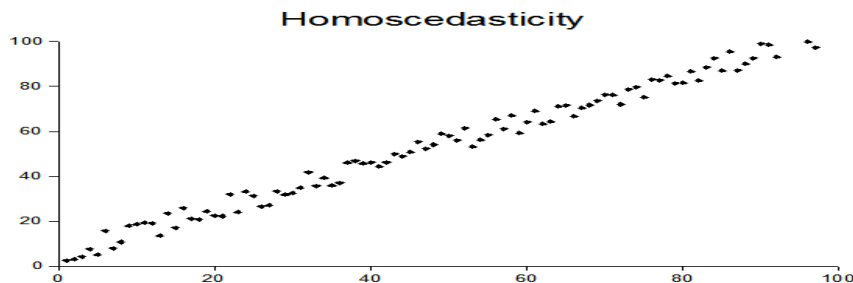
How to test:

Plot residuals vs. fitted values. Random scatter = ✓
Fan/funnel shape = ✗

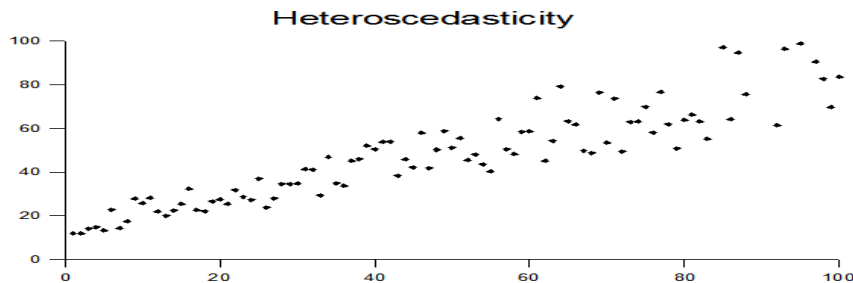
Fixes:

- Log-transform Y
- Weighted Least Squares
- Robust standard errors

✓ Homoscedastic



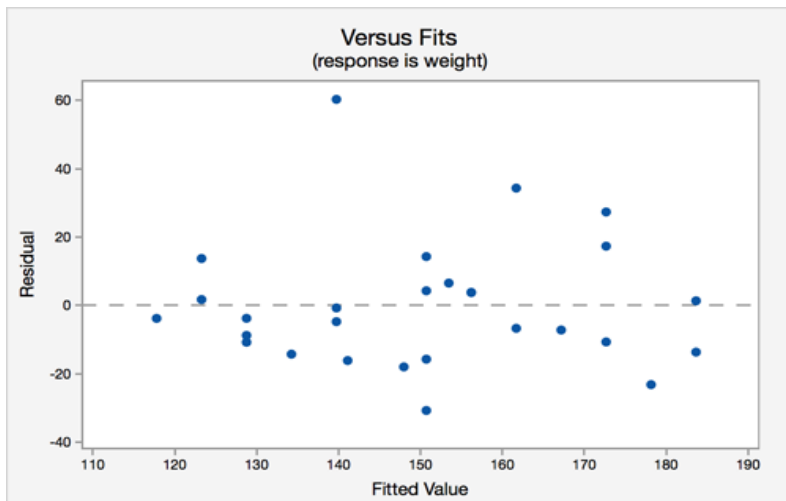
✗ Heteroscedastic



Assumptions 3 & 4 – Independence & Normality of Errors

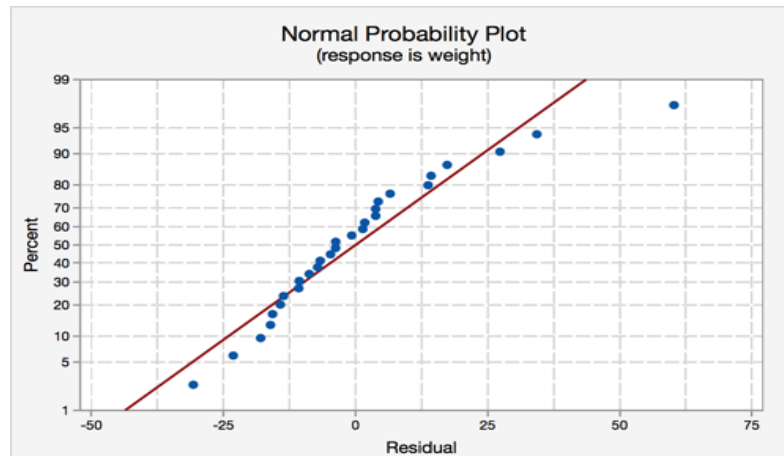
3. Independence of Errors

No relationship between residuals and fitted values.
Scatter must look random.

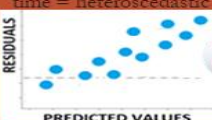


4. Normality of Errors

Residuals must be approximately normally distributed.
Check with a Normal Probability Plot — points should follow the diagonal line.



Assumptions Summary (excl. Multicollinearity)

What is it?	Assumption			
	Linearity	Independence of errors / Autocorrelation	Homoscedasticity	Normality
Test With	Expected value of DV is a linear function of each IV, holding the other variables fixed.	Error distribution is random and not influenced by or correlated with prior errors .	Errors have constant variance vs time, predictions, or any independent variable.	Data follows a normal distribution.
	Linearity = cluster around horizontal line)  PREDICTED VALUES	Pattern = autocorrelation  SEQUENTIAL TIME PERIOD Alternate DW Statistic	Residuals increase with time = heteroscedastic  PREDICTED VALUES Alternate: GQ Test	Normal probability plot of residuals  Alternate: Anderson-Darling test

 =Assumption met  =Assumption violated

Categorical Variables → Dummy Variables

Why Encode?

ML algorithms require numeric inputs.

State (NY / CA / FL) → binary columns.

⚠ Dummy Variable Trap

Including ALL k dummies causes perfect multicollinearity. Always drop one (reference category).

✓ **sklearn solution:**

```
from sklearn.preprocessing import OneHotEncoder

# drop='if_binary' avoids the dummy variable trap
enc = OneHotEncoder(drop='if_binary',
                    sparse=False)
X_enc = enc.fit_transform(X[['State']])
```

Before Encoding

R&D	State	Profit
165,349	New York	192,261
162,597	California	191,792
191,050	New York	183,502

After One-Hot Encoding

Florida = NY=0, CA=0 (reference group — implicit, avoids trap)

R&D	NY	CA	Profit
165,349	1	0	192,261
162,597	0	1	191,792
191,050	1	0	183,502
144,372	0	1	166,187

Assumption 5 – Multicollinearity

What is it?

Multicollinearity occurs when one predictor in a multiple regression model can be accurately predicted from the others. This inflates coefficient variances, making estimates unstable and hypothesis tests unreliable.

Correlation Matrix

Visualise pairwise linear relationships. Quick check but misses non-linear dependencies.

```
import seaborn as sns
sns.heatmap(df.drop('target', axis=1),
            cmap='cool', annot=True)

# With p-values:
corr, p = scipy.stats.spearmanr(df[f1], df[f2])
```

Variance Inflation Factor

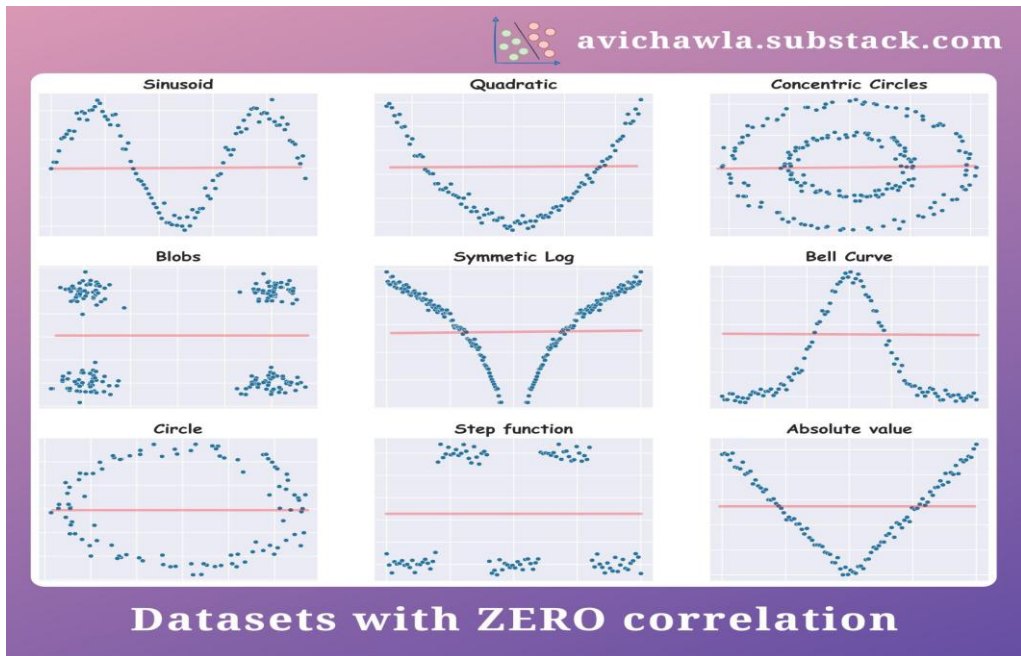
VIF quantifies how much a coefficient's variance is inflated. $VIF > 5-10$ = problem.

```
from statsmodels.stats.outliers_influence import
variance_inflation_factor

X = df.drop('target', axis=1)
vif = [variance_inflation_factor(X.values, i)
        for i in range(X.shape[1])]
print(dict(zip(X.columns, vif)))
```

Why Correlation Isn't Enough

Datasets with ZERO Pearson Correlation — but clear structure!



The Problem

Pearson correlation measures only *linear* relationships.

Non-linear patterns (quadratic, sinusoidal, concentric...) register as ~ 0 correlation even though X clearly predicts Y .

Sample size matters: different samples $\rightarrow$ different correlations.

Solution: use p-values to test significance AND use Predictive Power Score (PPS) for non-linear detection.

Predictive Power Score (PPS)

What is PPS?

$$\text{PPS} = 1 - (\text{Model_MAE} / \text{Naive_MAE})$$

Compares a model's MAE against a naïve baseline (predicting the mean).

- **PPS = 1:** X perfectly predicts Y
- **PPS = 0:** X is useless — no better than predicting the mean
- **Asymmetric:** $\text{PPS}(X \rightarrow Y) \neq \text{PPS}(Y \rightarrow X)$
- **Non-linear:** detects quadratic, step, and other non-linear patterns

PPS vs. Pearson Correlation

Aspect	Pearson	PPS
Direction	Bidirectional	Asymmetric (X→Y)
Non-linearity	✗ Misses it	✓ Detects it
Range	-1 to +1	0 to 1
Categorical targets	Limited	✓ Supported

```
import ppscore as pps

# Single score X→Y
pps.score(df, 'feature', 'target')

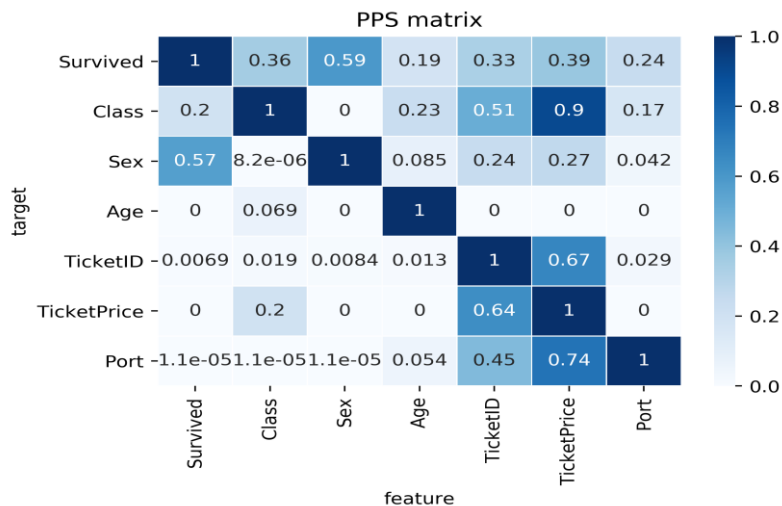
# Full matrix
pps.matrix(df)

# github.com/8080labs/ppscore
```

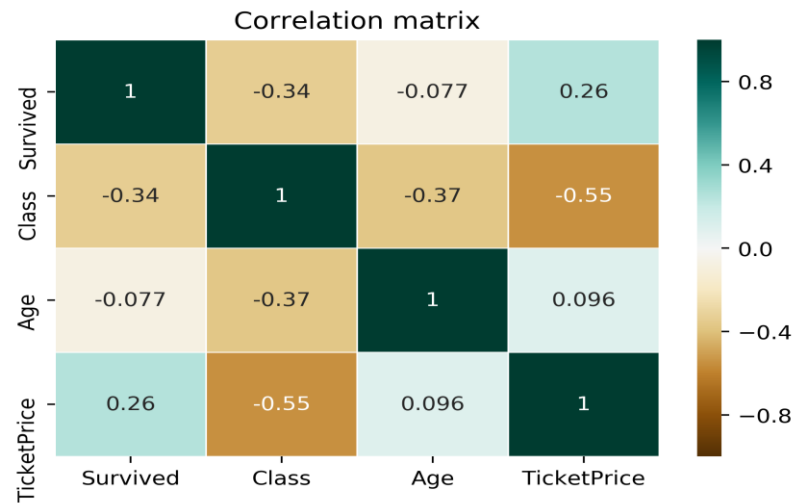
PPS Matrix vs. Pearson Matrix – Titanic Example

PPS Matrix

Sex → Survived: 0.57 (asymmetric — knowing sex helps predict survival)



Pearson Correlation Matrix



Part 2

Regularization & Loss Functions

Why Regularization?

The Bias–Variance Tradeoff

OLS minimises training error — but on high-dimensional or collinear data, coefficients can explode, leading to overfitting. Regularization adds a penalty term to shrink coefficients and improve generalisation.

LASSO (L1)

penalty: $\sum |\beta_j|$

Shrinks some to exactly 0 $\rightarrow$ sparse model / feature selection

Ridge (L2)

penalty: $\sum \beta_j^2$

Shrinks all toward 0 but never exactly $\rightarrow$ keeps all features

Elastic Net

penalty: $L1 + L2$

Combines both: sparse AND stable under collinearity

LASSO Regression – L1 Regularization

$$\min_w \|Y - w^T X\| + \alpha \cdot \sum |\beta_j|$$

What it does:

Adds an L1 (absolute value) penalty to the OLS loss. Shrinks the weight of uninformative features.

Key property:

Coefficients of irrelevant features are pushed exactly to zero → automatic feature selection.

Solves:

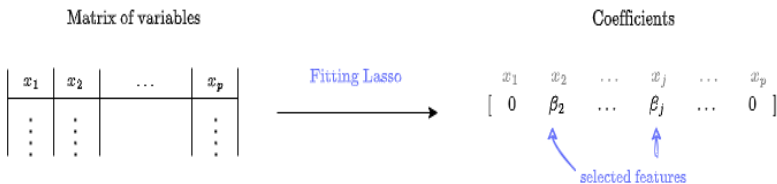
Multicollinearity by regularization — relaxes homoscedasticity requirement of OLS.

When to use:

Many features, most likely irrelevant. You want a sparse, interpretable model.

LASSO in Practice — Coefficient Shrinkage

How LASSO Selects Features



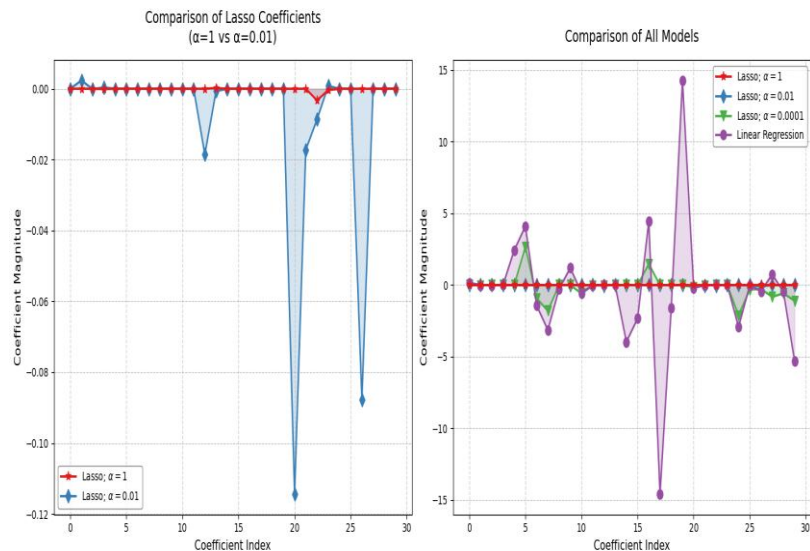
$x_1, x_p \rightarrow 0$ (irrelevant features eliminated)
 $\beta_2, \dots, \beta_j$ remain (selected features)

As α increases: more coefficients hit zero.
As $\alpha \rightarrow 0$: LASSO $\approx$ OLS (no penalization).

```
from sklearn.linear_model import Lasso
model = Lasso(alpha=0.1).fit(X_train, y_train)
```

Coefficient Analysis — Effect of α

Coefficient Analysis for Different Regression Models



Ridge Regression – L2 Regularization

$$\min_W \|Y - W^T X\| + \alpha \cdot \sum |w_j|^2$$

What it does:

Adds an L2 (squared) penalty to the OLS loss. Penalises large weights.

Key property:

Shrinks coefficients toward — but not to — zero. All features remain in the model.

Vs. OLS:

When multicollinearity inflates OLS variance, Ridge trades a little bias for much lower variance.

When to use:

All features believed to matter, but their coefficients may be inflated by collinearity.

Elastic Net & Regularization Summary

Elastic Net

- Yet another regression model
- Incorporates penalties from both L1 and L2 regularization

$$\frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2 + \lambda \left(\frac{1-\alpha}{2} \sum_{j=1}^m |w_j|^2 + \alpha \sum_{j=1}^m |w_j| \right)}{2n}$$

Shrinks some and eliminates some

$\alpha=1$ → pure LASSO · **$\alpha=0$** → pure Ridge

Shrinks some AND eliminates some.

λ and α tuned via cross-validation.

Best of both worlds:

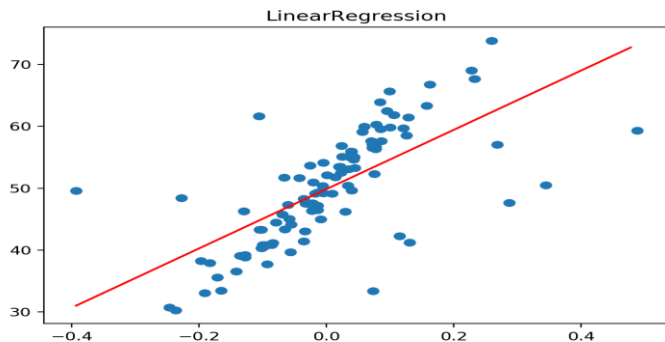
Sparsity + stability under collinearity.

Side-by-Side Comparison

	OLS	LASSO	Ridge	Elastic Net
Penalty	None	L1	L2	L1 + L2
Zeros coeff.	No	✓ Yes	Rarely	✓ Yes
Collinearity	✗ Unstable	Unstable	✓ Stable	✓ Stable
Feature sel.	No	✓ Built-in	No	✓ Built-in
sklearn	LinearRegression	Lasso(α)	Ridge(α)	ElasticNet(α , l1_ratio)

Robust Regression – Handling Outliers

The Problem with OLS



OLS minimises squared residuals → a single outlier can shift the fitted line dramatically.

Robust Alternatives

Huber Regressor

Squared loss for small residuals $|(y - X'w)/\sigma| < \epsilon$, absolute loss for large ones. Best of MSE + MAE.

RANSAC

Splits data into inliers and outliers, fits model only on inliers. Great for data with measurement errors.

Theil-Sen

Fits multiple models on subsets, combines medians. Single-median method; resistant to ~29% outliers.

Ref: towardsdatascience.com/dealing-with-outliers-using-three-robust-linear-regression-models

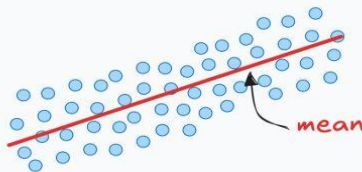
Beyond the Mean – Quantile Regression

Linear vs. Quantile

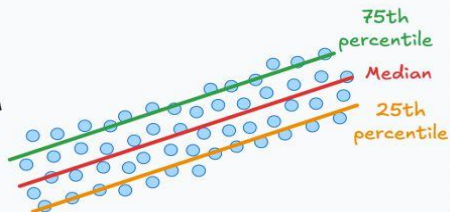
Which model do
you prefer?

 blog.DailyDoseofDS.com

Linear Regression

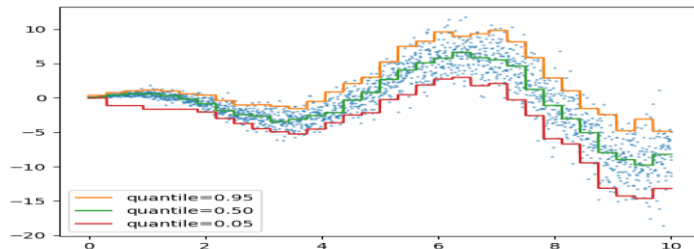


Quantile Regression



Code + Result

```
from sklearn.ensemble import  
HistGradientBoostingRegressor  
quantiles = [0.95, 0.5, 0.05]  
hist_q = {"q={q:.2f}":  
    HistGradientBoostingRegressor(  
        loss="quantile", quantile=q).fit(X, y)  
    for q in quantiles}
```



Regression Loss Functions – MSE, RMSE, MAE

MSE

Mean Squared Error

$$\sum (y_i - \hat{y}_i)^2 / n$$

Penalises large errors heavily. Differentiable → easy to optimise. Sensitive to outliers.

RMSE

Root Mean Sq. Error

$$\sqrt{\text{MSE}}$$

Same unit as Y. Preferred when large errors carry high cost. Dominated by outliers.

MAE

Mean Absolute Error

$$\sum |y_i - \hat{y}_i| / n$$

Robust to outliers. Linear penalty — all errors weighted equally. Less smooth to optimise.

R² and Adjusted R²

R² – Coefficient of Determination

$$R^2 = 1 - SS_{\text{res}} / SS_{\text{tot}}$$

Measures the amount of variance in Y that the model explains.

- 1 = perfect fit
- 0 = no better than predicting the mean
- Scale-free (always ≤ 1)

⚠ **Fatal flaw:** R² never decreases when you add predictors — even random noise improves it!



Adjusted R²

$$R^2_{\text{adj}} = 1 - (1 - R^2)(n - 1) / (n - k - 1)$$

n = sample size, k = number of predictors

Penalises for adding irrelevant features.

- Always lower than R²
- Increases only when new feature genuinely improves the model

✓ **Always prefer Adjusted R²** when comparing models with different numbers of predictors.

Part 3

Feature Selection

Less is More – Occam's Razor



"Entities should not be multiplied beyond necessity"

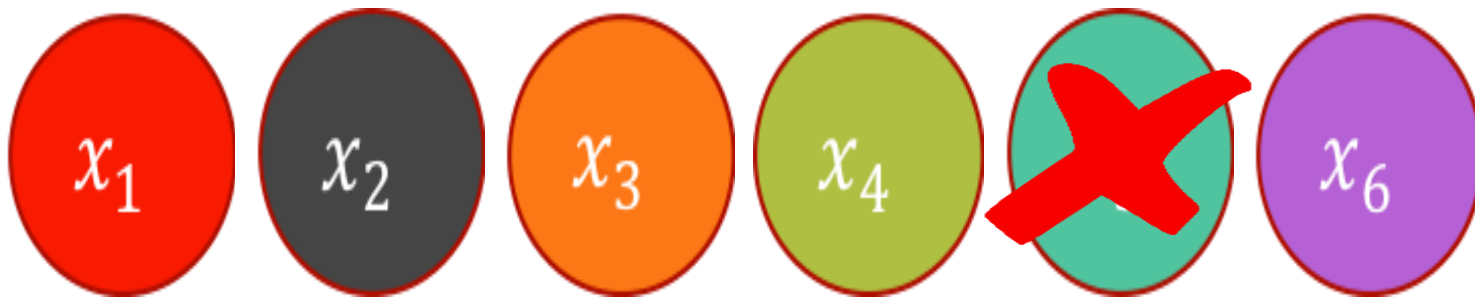
— William of Ockham (~1320)

Why does this matter for ML?

- ▶ **Curse of Dimensionality:** More features → exponentially more data needed to generalise well.
- ▶ **Overfitting:** Extra features can memorise training noise rather than learning the signal.
- ▶ **Compute & Memory:** High-dimensional data is slow and expensive to train.
- ▶ **Interpretability:** Simpler models with fewer features are easier to explain and debug.

Building a Model — Which Features to Include?

All features connected to the target — but which ones are actually useful?



Real-World High-Dimensional Datasets

- Gene expression: ~10,000 features (NCBI GEO)
- Proteomics: ~20,000 features (EBI PRIDE)
- Satellite imagery: Hundreds of spectral bands

The Core Challenge

Trying every possible subset is computationally infeasible at scale. We need smart search strategies.

The Combinatorial Explosion – Why Exhaustive Search Fails

Number of subsets of size k from N features
$$N! / (k! \cdot (N-k)!)$$

N features	k to select	# Subsets	Feasibility
100	5	75,287,520	⚠ Marginal
1,000	10	$\sim 2.6 \times 10^{23}$	✗ Infeasible
10,000	100	$\approx 5 \times 10^{35101}$	✗✗✗ Impossible
Universe	—	$\sim 10^{80}$ atoms	Reference

5×10^{35101}

$\approx$ the number of subsets when $N=10,000$ and $k=100$ — far more than atoms in the universe

5 Feature Selection Strategies

01

Score Comparison (All Possible)

Try every subset of k features and score each. Only feasible for very small N .

02

All-In

Include ALL features. Use when prior knowledge supports all predictors, or as a starting point for backward elimination.

03

Backward Elimination

Start with all features. Iteratively remove the least significant (highest p -value $> SL$) one at a time.

04

Forward Selection

Start with no features. Iteratively add the most significant (lowest p -value $< SL$) one at a time.

05

Bidirectional Elimination

Simultaneously applies forward and backward steps at each iteration. Most thorough wrapper method.

The P-Value — Foundation of Wrapper Methods

What is the p-value?

The p-value is the probability of obtaining a sample statistic at least as extreme as ours, assuming the null hypothesis ($H_0: \beta_j = 0$) is true.



Small p-value ($< SL$)

The feature's coefficient is unlikely to be zero by chance → it contributes meaningfully
→ KEEP

Large p-value ($> SL$)

The feature could easily have this coefficient by chance → it adds no value → REMOVE

Typical significance level (SL): 0.05 (95% confidence) | sometimes 0.01 for high-stakes applications

Backward Elimination – Step by Step

Algorithm

- Step 1** Set significance level SL (e.g. 0.05)
- Step 2** Fit model with ALL N predictors
- Step 3** Find predictor with highest p-value
- Step 4** If $p\text{-value} > SL \rightarrow$ remove that predictor
- Step 5** Refit model without removed predictor
- Step 6** Repeat Steps 3–5 until all p-values $< SL$

Example: 5 variables $\rightarrow$ 3 selected

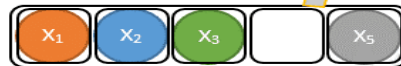
Backward stepwise selection example with 5 variables:

Start with a model that contains all the variables

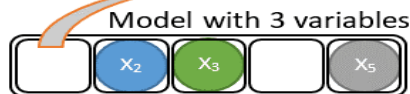


Remove the least significant variable

Model with 4 variables



Keep removing the least significant variable until reaching the stopping rule or running out of variables



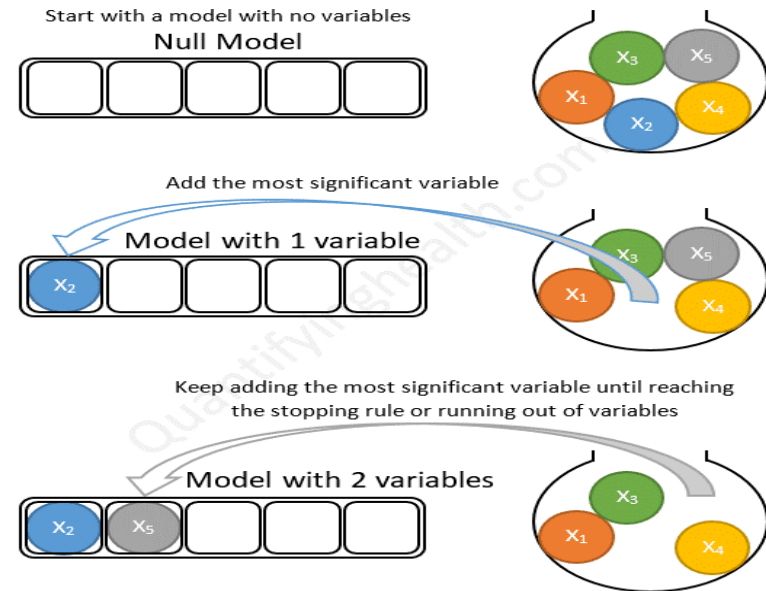
Forward Selection – Step by Step

Algorithm

- Step 1** Set significance level SL (e.g. 0.05)
- Step 2** Start with null model (no features)
- Step 3** Fit each remaining feature individually
- Step 4** Add the one with lowest p-value if $< SL$
- Step 5** Refit model with added predictor
- Step 6** Repeat until no remaining feature qualifies

Example: 5 variables $\rightarrow$ 2 selected

Forward stepwise selection example with 5 variables:



Bidirectional Elimination & All-In

Bidirectional Elimination

Combines forward selection and backward elimination simultaneously.

At each step: a new variable may be added (if $p < SL_enter$) AND one of the existing variables may be removed (if $p > SL_remove$).

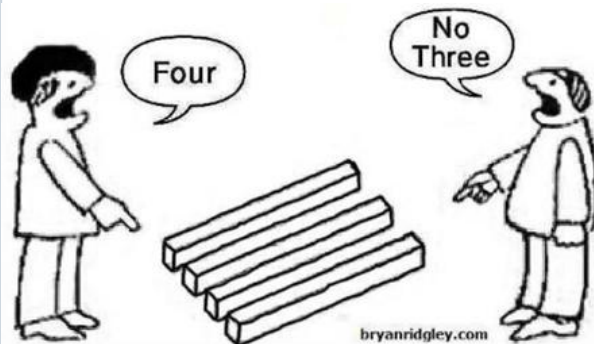
Advantage: more thorough than either alone — can undo sub-optimal early choices.

All-In

Include all available features without selection. Use when: (1) prior knowledge supports all predictors, (2) dataset is small and parsimonious models aren't needed, (3) preparing a baseline before backward elimination.

Models agree to disagree

Different selection methods applied to the same data can select different subsets. This is expected — they optimise different criteria.



Filter Methods — Statistical Feature Scoring

Filter methods score features independently of any model — fast and scalable to high-dimensional data.

Information Gain

Measures reduction in entropy when a variable is known. Higher IG
→ more useful for predicting the target.

```
from sklearn.feature_selection import SelectKBest,
mutual_info_classif
sel = SelectKBest(mutual_info_classif, k=10)
X_new = sel.fit_transform(X, y)
```

Chi-Square Test

Tests independence between categorical features and target. High χ^2
→ feature is not independent of target.

```
from sklearn.feature_selection import chi2,
SelectKBest
sel = SelectKBest(chi2, k=5)
X_new = sel.fit_transform(X_cat, y)
```

Correlation Coefficient

Measures linear relationship with target. Features should correlate
with target but not with each other.

```
# Pearson correlation
corr =
df.corr()['target'].abs().sort_values(ascending=False)
# Keep features above threshold
sel_feats = corr[corr > 0.3].index.tolist()
```

Advanced Feature Selection Strategies

F-Statistic

Ratio of between-group to within-group variance (ANOVA F-test). Tests whether a feature significantly explains variance in the target class distribution.

Mutual Information

Measures statistical dependence between variables — captures non-linear relationships that correlation misses. $MI(X;Y) = 0$ means X and Y are independent.

Boruta

Random Forest wrapper: iteratively compares each feature against a shuffled 'shadow' copy of itself. Features confirmed as better than their shadow are retained.

MRMR

Maximum Relevance Minimum Redundancy: selects features with maximum relevance to the target AND minimum pairwise redundancy with already-selected features.

Lesson 3 — Key Takeaways

Part 1

Linear Regression:

OLS fits a (hyper)plane to minimise squared residuals. Verify all 5 assumptions. Encode categoricals with one-hot; always drop one to avoid the dummy variable trap. Detect multicollinearity with correlation + VIF. Use PPS when non-linear relationships are suspected.

Part 2

Regularization:

L1 (LASSO) creates sparse models via feature zeroing. L2 (Ridge) stabilises estimates under multicollinearity. Elastic Net combines both. Choose MSE/RMSE when large errors are costly; MAE when robustness matters. Always report Adjusted R^2 for multi-feature model comparison.

Part 3

Feature Selection:

Exhaustive search is infeasible at scale. Backward elimination, forward selection, and bidirectional elimination use p-values as stopping criteria. Filter methods (MI, Chi^2) are fast and model-agnostic. Boruta and MRMR offer principled, reliable selection at high dimensionality.